IoT Akıllı Güvenlik ve Akıllı Fabrika Sistemi

Mimari çalışma rehberi: donanım, gömülü yazılım, yerel ağ, PC telemetri ve AWS IoT Core

Proje	`iot-guvenlik` (ESP32-S3 merkez + Arduino Nano oda düğümü)
Hazırlanma tarihi	22 Ekim 2026
Kaynak	Depodaki gerçek kod: `firmware/`, `libraries/RfPacket/`, `pc-dashboard/`, `cloud/`, `README.md`
Hedef okur	Bilgisayar mühendisliği 3. sınıf öğrencisi (çalışma ve sunum için)
Kapsam dışı	Şifre, anahtar ve sertifika içeren dosyalar (`secrets.h`, `aws_secrets.h`, `pc-dashboard/config.json`, `cloud/.certs/`) okunmadı ve alıntılanmadı.

Belgedeki her pin, süre, eşik, uç nokta, MQTT konusu ve JSON alanı koddan alınmıştır. Koddan kesin çıkarılamayan noktalar açıkça "belirsiz" ya da "kodda yok" diye işaretlenmiştir. Kablolamanın adım adım 3B anlatımı için ayrı belge: `docs/breadboard-rehberi.html`.

İçindekiler

1. Özet ve amaç
2. Genel mimari
3. Donanım
    1. Parçalar
    2. Pin tablosu
    3. Elektriksel kararlar
4. Merkez (hub) yazılım mimarisi
    1. Tasarım ilkeleri
    2. Sınıflar
    3. Bağımlılık diyagramı
    4. Ana döngü
    5. Kurulum portalı (bağlanmamış)
5. Alarm durum makinesi
6. 433 MHz RF protokolü
7. Oda düğümü (Nano)
8. Yerel web paneli
9. PC kontrol merkezi
0. AWS IoT Core bulut katmanı
    1. Kimlik ve bağlantı
    2. Konu yapısı
    3. IoT politikası
    4. IoT Rules ve DynamoDB
    5. Lambda kararı
    6. Device Shadow akışı
    7. Dağıtım betikleri
    8. Yedek ve maliyet
1. Teknoloji seçimleri tablosu
2. Donanım sorunları ve dersler

3. Sınav ve sunum soruları
4. Sözlük
5. Ek: Kodda gözlenen tutarsızlıklar

1 Özet ve amaç

Proje, bir odayı ya da küçük bir üretim alanını izleyen iki kartlı bir IoT sistemidir. **Arduino Nano** üzerindeki **oda düğümü** hareket (PIR), sıcaklık/nem (DHT11) ve gaz/duman (MQ-2) ölçer ve bunları 433 MHz radyo ile **ESP32-S3 merkeze** gönderir. Merkez kapı erişimini (RFID kart ve tuş takımı), alarm mantığını, servo ve röleyi yönetir; durumunu Wi-Fi üzerinden üç ayrı hedefe sunar: yerel web paneli, PC'deki telemetri paneli ve AWS IoT Core.

1.1 Sistemin işlevleri

İşlev	Nasıl yapılıyor	Ana dosya
Kart veya şifre ile giriş	RC522 kart UID'si `kAuthorizedCards` listesiyle, tuş takımı şifresi `Secrets::kDoorPin` ile karşılaştırılır. Onayda alarm kapanır, servo kapıyı 5 sn açar. 3 hatalı denemede giriş 30 sn kilitlenir.	`AccessControl.cpp`
Hareket alarmı	Sistem kuruluyken (Armed) düğümden gelen hareket bilgisi sireni başlatır.	`AlarmSystem.cpp`
Gaz alarmı	Sistem kurulu olmasa da çalışır; gaz seviyesi 100'e ulaşınca açılır, 80'e inince kapanır (histerezis).	`AlarmSystem.cpp`
433 MHz sensör ağı	Her okuma tek bir 32 bitlik rc-switch koduna sığdırılır; en fazla 4 düğüm (`kMaxNodes`), 15 sn sessiz kalan düğüm çevrimdışı sayılır.	`RfPacket.h` , `NodeMonitor.cpp`
Yerel web paneli	ESP32 üzerinde parola korumalı sayfa: `http://guvenlik.local` ; kur/çöz, kapı, röle, modül testi.	`WebPanel.cpp` , `web_page.h`
PC telemetri paneli	Merkez durum JSON'unu 5 sn'de bir PC'ye POST eder; Python sunucusu SQLite'a yazar, grafikli panel sunar.	`TelemetryClient.cpp` , `server.py`
AWS IoT Core bulut kararları	Telemetri MQTT/TLS ile buluta gider; bir Lambda sıcaklık/gaz eşiğine göre Device Shadow üzerinden fan (röle) ve vana (servo) komutu verir.	`CloudLink.cpp` , `cloud/`
Modül testi	`B` tuşu + kart/şifre (veya paneldeki düğme): servo açılıp kapanır, 2 sn bekler, röle 10 sn açık kalır, LED'ler kayan ışıkla yanar.	`TestSequence.cpp`
Durum LED'leri	Kırmızı gaz alarmı, sarı hareket, mavi sıcaklık >= 30 °C. Hem merkezde hem düğümde aynı renk düzeni.	`IndicatorLeds.h` , `room_node.ino`
Telegram bildirimi	Hazır ama varsayılan olarak kapalı; token ve chat id boşsa hiçbir şey göndermez.	`TelegramNotifier.cpp`

1.2 Akıllı fabrika yorumu

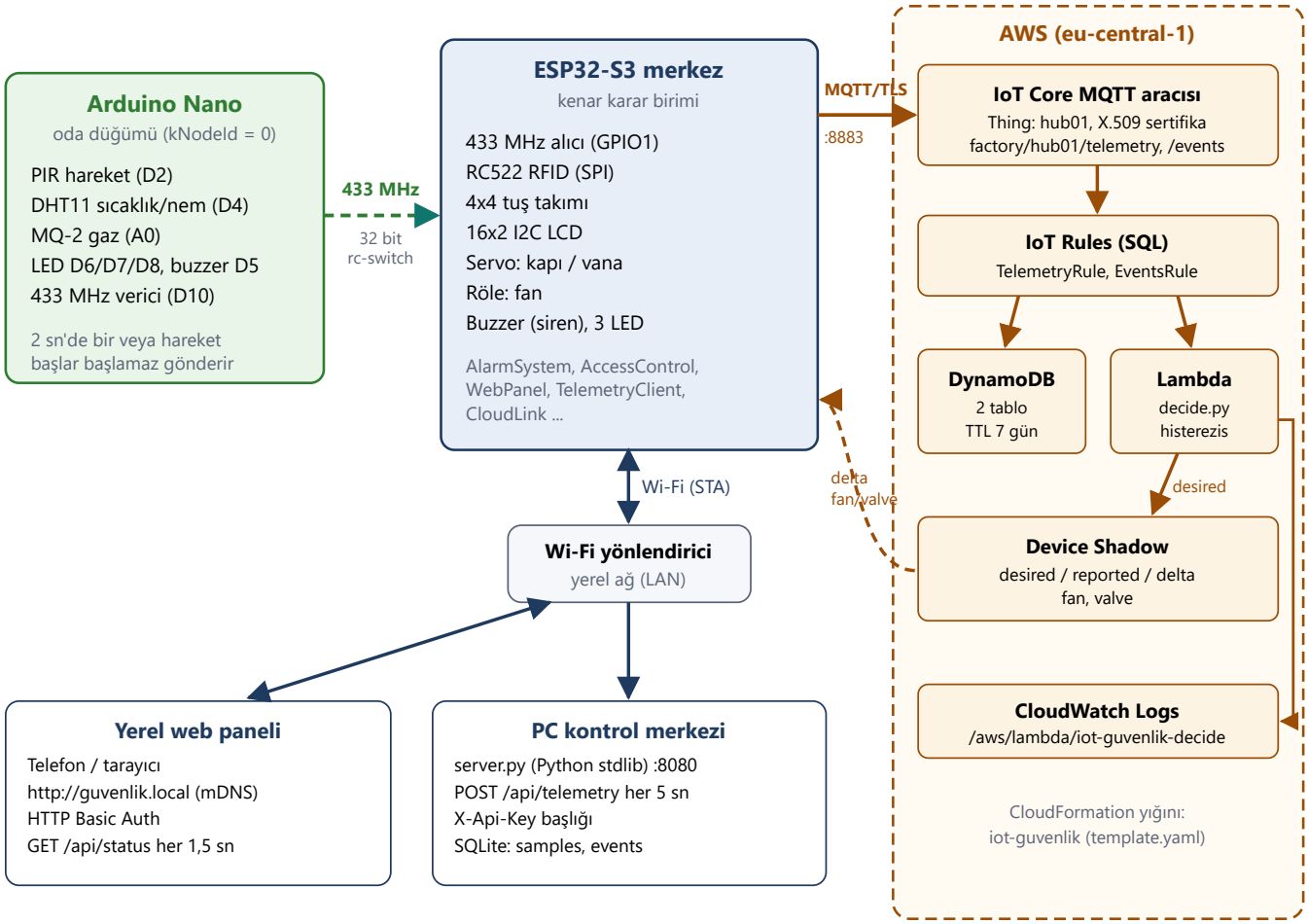
Aynı donanım bir üretim hücresi olarak da okunabilir. Bulut kodu bu yorumu açıkça kullanır: `template.yaml` açıklaması "smart-factory hub", MQTT konu öneki `factory` , Lambda komutları `fan` ve `valve` adını taşır.

Bileşen	Güvenlik yorumu	Fabrika yorumu
Röle (GPIO47)	Lamba, siren ışığı	**Havalandırma fanı** (Shadow alanı `fan` : `on` / `off`)
Servo (GPIO21)	Kapı mandalı	**Gaz vanası / havalandırma kapağı** (Shadow alanı `valve` : `open` / `closed`)
MQ-2 + DHT11	Yangın/duman uyarısı	Ortam gazı ve ısı izleme
RFID + tuş takımı	Kapı girişi	Yetkili personel girişi

Neden bu yorum? Bulut katmanı "algıla - karar ver - eyleme geçir" döngüsünü göstermek için kurulmuştur. Tehlike algılanınca fanı çalıştırıp kapağı açmak, endüstriyel IoT'nin tipik kapalı döngü kontrol örneğidir ve mevcut iki aktüatörle ek donanım gerektirmeden gösterilebilir.

2 Genel mimari

Sistem üç katmanlıdır: **algılama katmanı** (Nano oda düğümü), **kenar (edge) katmanı** (ESP32-S3 merkez; yerel kararları kendisi verir) ve **uygulama/bulut katmanı** (yerel panel, PC paneli, AWS). Kritik güvenlik kararları (alarm, siren, kilit) kenarda verilir; bulut yalnızca ek, gecikmeye dayanıklı kararlar (fan/vana) üretir.



Şekil 1. Genel mimari. Yeşil kesikli ok: radyo; lacivert: yerel HTTP; turuncu: AWS (MQTT/TLS internet üzerinden, yerel yönlendiriciden geçer).

2.1 Veri yolları özetle

Yol	Protokol	Yön ve sıklık	Kaynak
Nano → ESP32	433 MHz OOK, rc-switch, 32 bit	Tek yönlü; 2 sn'de bir veya hareket başlangıcında, her kod 4 kez tekrarlanır	`room_node/config.h`
Tarayıcı ↔ ESP32	HTTP/1.1, Basic Auth, JSON	Sayfa 1,5 sn'de bir `/api/status` sorar	`web_page.h`
ESP32 → PC	HTTP POST, JSON, `X-Api-Key`	5 sn'de bir; hata olursa 30 sn bekler	`config.h` `Telemetry`
ESP32 ↔ AWS	MQTT 3.1.1 / TLS (port 8883), X.509 karşılıklı kimlik	Telemetri 5 sn'de bir; olaylar oluştukça; Shadow delta buluttan gelir	`config.h` `Cloud`, `CloudLink.cpp`

> **Neden üç ayrı çıkış?** Her biri farklı bir ihtiyaca cevap verir ve birbirinden bağımsız çalışır: yerel panel internet ve PC olmadan da çalışan **yedek** arayüzdür; PC paneli **geçmiş veri ve grafik** sağlar; AWS **uzaktan erişim, kalıcı kayıt ve bulut kararı** sağlar. Birinin çökmesi ötekileri etkilemez (kısa zaman aşımları ve geri çekilme sayesinde).

3 Donanım

3.1 Parçalar

Parça	Kart	Görev	Not
ESP32-S3 (DevKitC-1 pin düzeni; YD-ESP32-S3 klonu kullanılmış)	Merkez	Ana işlemci, Wi-Fi	USB'den beslenir
Arduino Nano (ATmega328P)	Düğüm	Sensör okuma, RF gönderim	Gerekirse "Old Bootloader" seçimi
RC522 RFID okuyucu (13,56 MHz)	Merkez	Kart UID okuma	SPI, 3,3 V
4x4 membran tuş takımı	Merkez	Şifre, `A` kur, `B` test, `*` sil, `#` onay	Şerit ters takılı, yazılımda düzeltilmiş
16x2 LCD + I2C sırt kartı	Merkez	Durum ve mesajlar	Adres 0x27 (olmazsa 0x3F)
SG90 tipi servo	Merkez	Kapı mandalı / vana	0° kapalı, 90° açık
Tek kanallı 5 V röle modülü (aktif düşük)	Merkez	Fan / lamba	Open-drain sürülür
Aktif buzzer + BC547 + 1 kΩ	Merkez	Bip ve siren	Transistörle sürülür
433 MHz alıcı + 4,7 kΩ / 10 kΩ bölücü	Merkez	RF paket alma	17,3 cm anten teli önerilir
433 MHz verici	Düğüm	RF paket gönderme	
PIR (HC-SR501 tipi), DHT11, MQ-2	Düğüm	Hareket, sıcaklık/nem, gaz	MQ-2 60 sn ısınır
Kırmızı/sarı/mavi LED + 220 Ω	İkisi de	Gaz / hareket / sıcaklık göstergesi	pin → uzun bacak, kısa bacak → 220 Ω → GND
MB102/HW-131 breadboard güç modülü + 12 V adaptör	Breadboard	Servo, röle, LCD, RF için 5 V	Bkz. 3.3

Parça modelleri (SG90, HC-SR501) koddan değil, breadboard rehberi ve README'den çıkarılmıştır; kodda yalnızca pin ve davranış tanımlıdır.

3.2 Pin tablosu

Merkez: `firmware/security_hub/config.h` (`Config::Pin`)

Sinyal	GPIO	Sabit
RC522 SS (SDA)	10	`kRfidSs`
RC522 MOSI	11	`kRfidMosi`
RC522 SCK	12	`kRfidSck`
RC522 MISO	13	`kRfidMiso`
RC522 RST	14	`kRfidRst`
LCD SDA (I2C)	8	`kLcdSda`
LCD SCL (I2C)	9	`kLcdScl`
Servo sinyali	21	`kServo`
Buzzer (BC547 tabanı)	2	`kBuzzer`
Röle IN	47	`kRelay`
433 MHz alıcı DATA (bölücü üzerinden)	1	`kRfReceiver`
LED kırmızı (gaz)	38	`kLedGas`
LED sarı (hareket)	39	`kLedMotion`
LED mavi (sıcaklık)	40	`kLedTemp`
Tuş takımı satırları	18, 17, 16, 15	`keypadRows`
Tuş takımı sütunları	7, 6, 5, 4	`keypadCols`

Düğüm: `firmware/room_node/config.h`

Sinyal	Pin	Sabit
PIR çıkışı	D2	`kPir`
DHT11 veri	D4	`kDht`
Buzzer	D5	`kBuzzer`
433 MHz verici DATA	D10	`kRfTransmitter`
Durum LED'i (gönderimde yanar)	LED_BUILTIN (D13)	`kStatusLed`
MQ-2 analog çıkış	A0	`kGas`
LED kırmızı (gaz)	D6	`kLedGas`
LED sarı (hareket)	D7	`kLedMotion`
LED mavi (sıcaklık)	D8	`kLedTemp`

Diğer donanım sabitleri

Ayar	Değer
LCD adresi / boyut	0x27, 16x2
Servo kapalı / açık	0° / 90°, 50 Hz
Röle	`kActiveLow = true`
DHT tipi	DHT11
Düğüm kimliği	`kNodeId = 0` (0..3)

> **Tuş takımı notu:** `config.h` yorumuna göre şerit kablo bu yapımda ters takılmıştır; ölçülen tuş haritasına göre satır/sütun pinleri yer değiştirmiş ve ters sıralanmıştır. Donanımı sökmek yerine yazılımda pin dizisinin sırası değiştirilmiştir.

3.3 Elektriksel kararlar ve nedenleri

Karar	Neden
RC522 ESP32'nin 3V3 pininden beslenir	RC522 3,3 V'luk bir modüldür; 5 V hattına bağlanırsa bozulur. Lojik seviyesi de ESP32 ile aynı (3,3 V) olduğu için seviye dönüştürücü gerekmez. ESP32'nin 3V3 pininden yalnızca RC522 beslenir.
RF alıcı çıkışında 4,7 kΩ / 10 kΩ gerilim bölücü	Alıcı 5 V ile beslenir ve 5 V'luk sinyal verir; ESP32 pinleri 5 V toleranslı değildir. Bölücü çıkışı: V_{out} = 5 V × 10k / (4,7k + 10k) ≈ **3,40 V**. Bu değer ESP32'nin HIGH eşiğinin üstünde, mutlak sınırının altındadır.
Buzzer BC547 NPN transistörle sürülür (taban direnci 1 kΩ)	GPIO pininin akım kapasitesi sınırlıdır ve buzzer 5 V ile daha yüksek sesli çalışır. Transistör anahtar görevi görür: pin yalnızca küçük taban akımı verir, buzzer akımı 5 V hattından gelir. 1 kΩ taban akımını sınırlar.
Röle pini open-drain (`OUTPUT_OPEN_DRAIN`)	Aktif düşük 5 V röle modülünü kapatmak için girişinin modülün kendi VCC'sine (5 V) çekilmesi gerekir; ESP32'nin 3,3 V HIGH seviyesi yetmez, röle yarı açık kalabilir. Open-drain modda pin "kapalı" durumda bırakılır (yüksek empedans) ve modülün kendi pull-up direnci girişi 5 V'a çeker. "Açık" durumda pin LOW'a çeker.
Ortak GND	ESP32 USB'den, servo/röle/LCD güç modülünden beslenir. Sinyallerin anlamlı olması için tüm toprakların birleşmesi gerekir; aksi halde referans gerilimi tanımsız kalır ve sinyaller okunamaz.
Güç modülünde iki jumper da **5V** konumunda	Modülün iki ucundaki "5V OFF 3.3" başlıkları 3,3 V konumundayken LCD bozuk yazar, RF alıcı ve servo çalışmaz (README). Bu modüller 5 V ister.
12 V adaptör (README ve breadboard rehberi: "en az 2A")	Servo hareket anında yüksek akım çeker; röle bobini de akım ister. 9 V pil veya 700 mA USB kablosu servo ile röleyi birlikte taşıyamaz, gerilim çöker ve ESP32 resetlenebilir ya da servo yalnızca tıkırdar. Bu kurulumda 3 A'lik adaptör kullanıldığı belirtilmiştir; koddaki belgelerde alt sınır 2 A yazar.
Breadboard kenar hatlarında köprüler (29-34 arası)	830 delikli breadboard'ların çoğunda güç hatları ortadan ikiye bölünür. Köprü olmadan güç modülünün 5 V'u diğer yarıdaki modüllere ulaşmaz. Dört hatta da köprü takılır.
Servo darbeleri hareketten 700 ms sonra durdurulur	`kServoSettleMs` : hareket bitince `servo_.release()` çağrılır; servo sürekli PWM aldığında titreyip vızıldar ve boşa akım çeker.
LED'ler 220 Ω ile	Akım sınırlama; LED'i ve pini korur.

4 Merkez (hub) yazılım mimarisi

Merkez yazılımı Arduino C++ ile, her donanım veya sorumluluk için ayrı bir sınıf olacak şekilde yazılmıştır. `security_hub.ino` bir **kompozisyon kökü**dür: tüm nesneleri global olarak oluşturur, aralarındaki bağlantıları (referanslar ve lambda fonksiyonları) kurar, `setup()` içinde başlatır ve `loop()` içinde sırayla günceller.

4.1 Tasarım ilkeleri

İlke	Projede nasıl uygulanıyor	Neden
Tek sorumluluk (SRP)	`Buzzer` yalnızca ses çıkarır, `AlarmSystem` yalnızca alarm mantığını bilir, `StatusReport` yalnızca JSON üretir.	Her sınıf tek başına anlaşılır ve test edilir; değişiklik tek dosyada kalır.
millis() tabanlı bloklamayan döngü	Hiçbir sınıf `delay()` kullanmaz (yalnızca açılıştaki Wi-Fi bağlantısı 250 ms adımlarla bekler). Zamanlamalar "bitiş zamanı" saklanarak yapılır: `beepUntil_`, `closeAt_`, `flashUntil_`, `stepEnds_`.	Tek çekirdekli düşünülen döngüde kart okuma, RF alma ve web sunucusu aynı anda yanıt verebilmelidir. 5 sn'lik bir `delay()` bu sürede gelen RF paketlerini ve tuşları kaçırırdı.
Taşmaya dayanıklı zaman karşılaştırma	`static_cast<int32_t>(millis() - hedef) >= 0` kalıbı.	`millis()` yaklaşık 49,7 günde taşar; işaretli fark kalıbı taşmada da doğru çalışır.
Merkezi `config.h`	Tüm pinler, süreler, eşikler, LCD metinleri ve olay mesajları tek dosyada, `Config::` ad alanları altında.	Ayar değiştirmek için kod içinde arama gerekmez; "sihirli sayı" yoktur.
Gizli bilgi ayrımı	Wi-Fi şifresi, kapı şifresi, panel şifresi, telemetri anahtarı `secrets.h` içinde (git'e girmez). Şablonu `secrets.example.h`. AWS sertifikaları `aws_secrets.h` içinde, `deploy.ps1` üretir.	Şifreler depoya sızmaz; `SecretsLoader.h` dosya yoksa anlaşılır bir derleme hatası verir.
EventSink ile gevşek bağlılık	Sınıflar olayları doğrudan log'a ya da Telegram'a yazmaz; kendilerine verilen `EventSink` fonksiyonunu çağırır.	Olayın nereye gideceği (log, Telegram, ileride başka bir yer) tek noktada, `report()` içinde değişir.
Bağımlılık enjeksiyonu	Bağımlılıklar yapıcı (constructor) parametresi olarak verilir: `AlarmSystem(Buzzer&, EventSink)`, `WebPanel(Deps)`, `CloudLink(..., Actuators)`.	Sınıflar global değişken aramaz; kimin kimi kullandığı yapıcıdan okunur.

firmware/security_hub/EventSink.h

```
using EventSink = std::function<void(const String& message, bool urgent)>;
```

firmware/security_hub/security_hub.ino

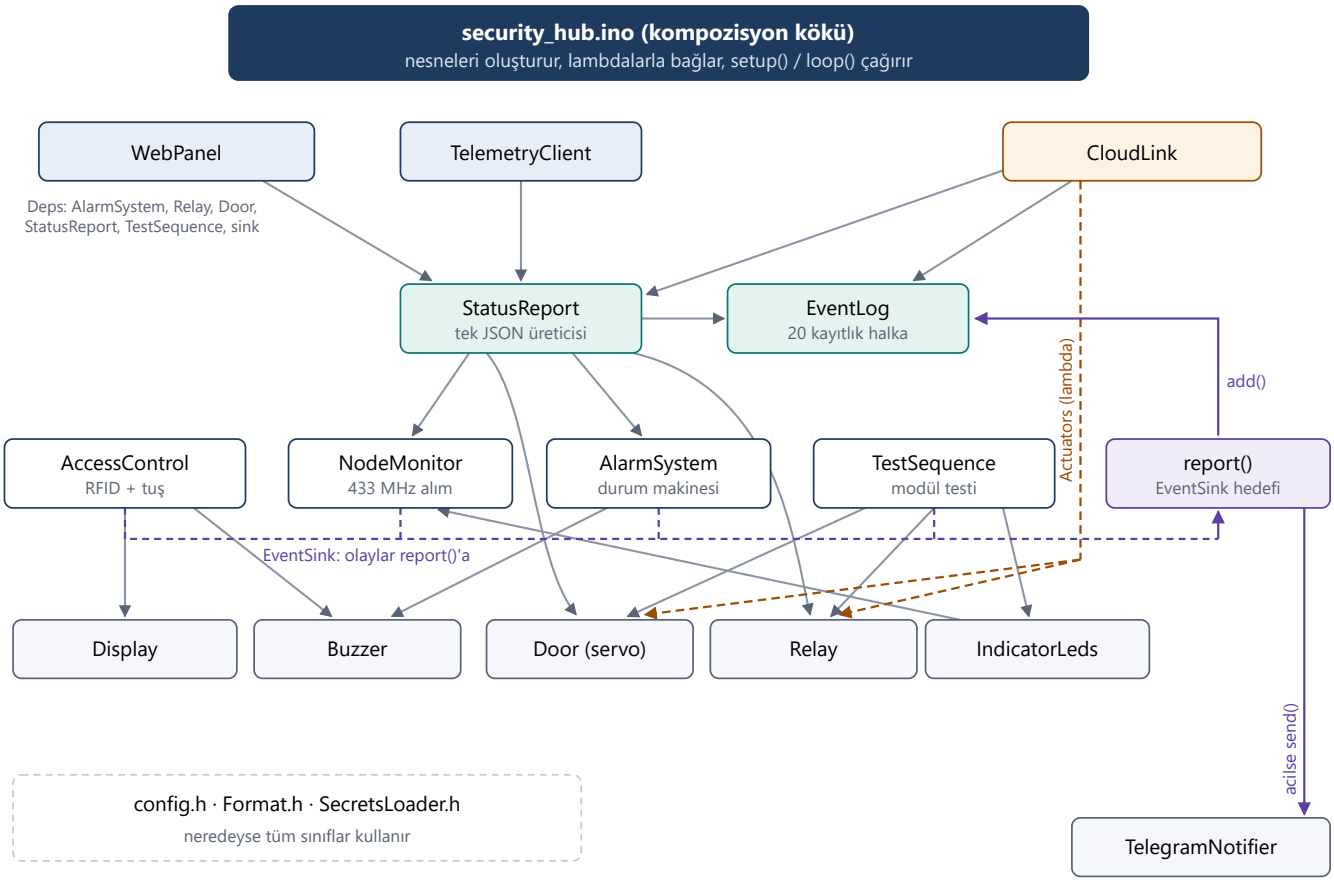
```
void report(const String& message, bool urgent) {
  eventLog.add(message, urgent);
  if (urgent) {
    notifier.send(message);
  }
}
```

4.2 Sınıflar ve dosyalar

Sınıf / dosya	Sorumluluk	Etkileşim
`AccessControl` .h / .cpp	RC522 kartını ve tuş takımını yoklar; şifre tamponunu tutar (en fazla 8 hane, LCD'de `*` ile maskeli); 3 hatada 30 sn kilit uygular. Tanımsız kart UID'sini olay olarak yazar.	`Display`, `Buzzer` kullanır; `Handlers` yapısıyla dışarıya `onGranted`, `onArmRequest`, `onTestRequest`, `onEvent` bildirir.
`AlarmSystem`	Disarmed/Arming/Armed/Triggered durum makinesi ve her zaman açık gaz alarmı (histerezisli). Siren kararını verir.	`Buzzer::setSiren()`; olaylar `EventSink`'e.
`Door`	Servo mandal: `open()` 5 sn açıp kendiliğinden kapatır; `hold(bool)` bulut vana komutu için süresiz açık/kapalı tutar. Hareket sonrası 700 ms'de PWM'i bırakır.	`AccessControl` (main üzerinden), `WebPanel`, `TestSequence`, `CloudLink`.
`Relay` (yalnız .h)	Röle çıkışı; aktif düşük modül için open-drain.	`WebPanel`, `TestSequence`, `CloudLink`.
`Buzzer`	Bloklamayan aktif buzzer: kısa bip (tuş 40 ms, onay 150 ms, ret 800 ms) ve 250 ms aralıkla açılıp kapanan siren.	`AccessControl`, `AlarmSystem`.
`Display`	16x2 I2C LCD: kalıcı durum ekranı + 2,5 sn'lik geçici mesaj (`flash`). Aynı içerik yeniden çizilmez (titreme olmaz).	`AccessControl`, main (`setStatus` her 1 sn).
`EventLog`	20 kayıtlık halka tampon (`kLogCapacity`); zaman damgası NTP varsa `SS:DD:ss`, yoksa `+Ns`. `total()` eklenen tüm olayların sayısını tutar.	`report()` yazar; `StatusReport` ve `CloudLink` okur.
`NodeMonitor`	rc-switch ile 433 MHz kesme tabanlı alım; bit uzunluğu, checksum, sihirli sayı ve düğüm kimliği doğrulanır; 15 sn sessiz düğüm çevrimdışı.	`RfPacket::decode`; olaylar `EventSink`'e; `IndicatorLeds`, `StatusReport` okur.
`StatusReport`	Durumu tek bir JSON belgesine çevirir (ArduinoJson). Aynı JSON yerel panel, PC ve AWS'ye gider.	`AlarmSystem`, `NodeMonitor`, `EventLog`, `Relay`, `Door` okur.
`WebPanel`	Port 80'de `WebServer`; her rota Basic Auth ile korunur; API rotaları işlemden sonra güncel durumu döner.	`Deps`: alarm, röle, kapı, rapor, test, sink.
`TelemetryClient`	Durum JSON'unu PC'ye POST eder; başarıda 5 sn, hatada 30 sn sonra tekrar dener; zaman aşımı 800 ms.	`StatusReport`; `HTTPClient`.
`CloudLink`	AWS IoT Core MQTT/TLS bağlantısı; telemetri ve yeni olayları yayınlar; Shadow delta mesajlarını uygular ve `reported` durumu bildirir.	`StatusReport`, `EventLog`; `Actuators` lambdaları ile `Relay` ve `Door`.
`TestSequence`	Modül testi adımları: AwaitAuth → Servo → Gap → Relay → Leds; bloklamadan ilerler.	`Door`, `Relay`, `IndicatorLeds`, `EventSink`.
`IndicatorLeds` (yalnız .h)	Üç durum LED'i: gaz alarmı, herhangi bir çevrimiçi düğümde hareket, sıcaklık >= 30 °C. Test için `showOnly(i)`.	`NodeMonitor` okur.
`TelegramNotifier`	Bot API'ye HTTPS POST. Token/chat id boşsa devre dışı. Sertifika doğrulaması kapalı (`setInsecure()`, "demo project"). Mesaj başına yaklaşık 1 sn bloklar.	Yalnızca `report()` acil olaylarda çağırır.
`Format.h`	`format(fmt, ...)`: printf benzeri, 160 baytlık tamponla `String` üretir.	Config mesajlarındaki `%s`, `%u` yer tutucuları için.
`SecretsLoader.h`	`__has_include("secrets.h")` ile dosyayı ekler; yoksa Türkçe açıklamalı `#error`.	Şifre kullanan her .cpp.
`config.h`	Pin, zamanlama, güvenlik, telemetri, bulut, saat dilimi, ağ, kurulum, yetkili kartlar, LCD metinleri, mesajlar.	Hemen her dosya.

`web_page.h`	Yerel panelin HTML/JS'i, `PROGMEM` içinde ham dize olarak.	`WebPanel` `"/"` rotası.
`DeviceSettings`, `SetupPortal`, `setup_page.h`	NVS'de ayar saklama ve telefondan Wi-Fi/AWS ayarı için erişim noktası + captive portal.	Dosyalar mevcut ama `security_hub.ino` içinde kullanılmıyor (bkz. 4.5).

4.3 Bağımlılık diyagramı



Gri ok: "kullanır" (yapıcıda verilen referans). Turuncu kesikli: CloudLink'in Actuators lambdaları (fan → Relay, vana → Door).
Mor: AccessControl, NodeMonitor, AlarmSystem, TestSequence ve WebPanel olayları report()'a verir; o da EventLog'a yazar, acilse Telegram'a yollar.

Şekil 2. Merkez sınıflarının bağımlılıkları. Okunabilirlik için WebPanel'in bağımlılıkları ok yerine kutunun altında listelendi.

> **Neden lambda ile bağlantı?** `AccessControl`, kapı açılınca ne olacağını bilmez; yalnızca `onGranted(who, method)` çağırır. main içindeki lambda alarmı kapatır, olayı yazar ve test bekleniyorsa testi, değilse kapıyı başlatır. Böylece `AccessControl` ile `AlarmSystem` / `TestSequence` arasında doğrudan bağımlılık oluşmaz. Aynı şekilde `CloudLink` "fan" ve "vana" kavramlarıyla çalışır; bunların röle ve servoya karşılık geldiği yalnızca main'de yazılıdır.

`firmware/security_hub/security_hub.ino`

```
CloudLink cloud(statusReport, eventLog, {
    [](bool on) { relay.set(on); },     // setFan
    [](bool open) { door.hold(open); }, // setValve
    [] { return relay.isOn(); },        // fanOn
    [] { return door.isOpen(); },       // valveOpen
});
```

4.4 Açılış ve ana döngü

setup(): seri port (115200) → buzzer, röle, LED, kapı, LCD, erişim kontrolü, RF alıcı başlatılır → Wi-Fi'ye en fazla 15 sn (`kWifiConnectTimeoutMs`) bağlanmaya çalışılır. Başarılıysa NTP saati (`pool.ntp.org`, saat dilimi `<+03>-3`), web paneli,

mDNS (`guvenlik` , http/tcp/80) başlar ve IP LCD'de gösterilir. Başarısızsa "çevrimdışı devam ediliyor" olayı yazılır; yerel güvenlik işlevleri yine çalışır.

loop() her turda sırayla:

1. `buzzer.update()` , `door.update()` , `display.update()` : zaman dolan bip/kapı/mesajları bitirir.
2. `accessControl.update()` : kart ve tuş yoklaması.
3. `alarmSystem.update()` : Arming süresi dolduysa Armed'a geçer.
4. `nodeMonitor.update()` : sessiz düğümleri çevrimdışı yapar.
5. `nodeMonitor.poll(data)` : geçerli paket geldiyse `onMotion()` ve `onGasLevel()` .
6. `testSequence.update()` ; test çalışmıyorsa `indicators.update()` .
7. `panel.handle()` , `telemetry.update()` , `cloud.update()` : ağ işleri.
8. Her 1 sn (`kLcdRefreshMs`) LCD durum satırları yenilenir.

> **Neden bu sıra?** Güvenlik açısından kritik işler (giriş, alarm, RF) ağ işlerinden önce gelir. Ağ işleri kısa zaman aşımlarıyla sınırlandırılmıştır (PC için 800 ms), böylece en kötü durumda bile döngü hızlı döner.

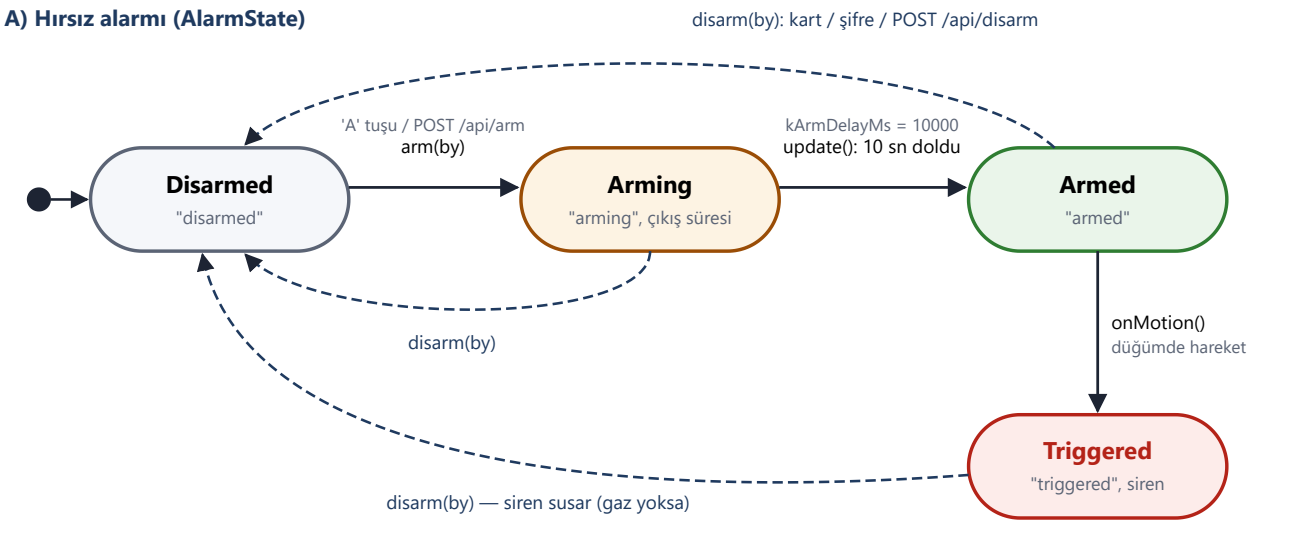
4.5 Kurulum portalı (kodda var, bağlanmamış)

`DeviceSettings` Wi-Fi, AWS uç noktası, thing adı, konu öneki, üç PEM sertifika alanı ve telemetri URL'sini ESP32'nin kalıcı belleğinde (NVS, `Preferences` , ad alanı `settings`) saklar; kayıt yoksa derleme zamanı değerlerine (secrets.h, aws_secrets.h, config.h) düşer. `SetupPortal` `Guvenlik-Kurulum-XXXX` adlı bir erişim noktası (192.168.4.1), DNS ile "captive portal" yönlendirmesi, `/setup` , `/setup/scan` , `/setup/save` rotaları sunar; PEM alanlarını `-----BEGIN` / `-----END` varlığıyla doğrular, kaydettikten 2 sn sonra yeniden başlar. `config.h` içinde bununla ilgili sabitler (`Timing::kSetupWindowMs` = 180 sn, `Keypad4x4::kSetup = 'D'` , `Setup::`) bulunur.

> **Durum:** Mevcut `security_hub.ino` bu sınıfları oluşturmuyor; Wi-Fi doğrudan `Secrets::kWifiSsid` ile, AWS doğrudan `aws_secrets.h` ile yapılandırılıyor ve `AccessControl` `D` tuşunu işlemiyor. Bu özellik yarım kalmış ya da ayrı bir dalda geliştirilen bir çalışma gibi görünüyor. Sunumda "planlanan özellik" olarak anlatılmalıdır. Ayrıca bkz. Ek, madde 1-2.

5 Alarm durum makinesi

`AlarmSystem` iki bağımsız mantık içerir: **hırsız alarmı** için dört durumlu bir sonlu durum makinesi (`enum class AlarmState { Disarmed, Arming, Armed, Triggered }`) ve **gaz alarmı** için iki durumlu histerezisli bir bayrak (`gasAlarm_`). Siren bu ikisinin VEYA'sıdır.



Şekil 3. Alarm durum makinesi ve gaz histerezisi. Siren = (state == Triggered) VEYA gasAlarm_ (`refreshSiren()`).

`firmware/security_hub/AlarmSystem.cpp`

```
void AlarmSystem::onGasLevel(uint8_t level) {
  if (!gasAlarm_ && level >= Config::Security::kGasAlarmLevel) {        // 100
    gasAlarm_ = true;  refreshSiren(); sink_(format(Config::Msg::kGas, level), true);
  } else if (gasAlarm_ && level <= Config::Security::kGasClearLevel) {  // 80
    gasAlarm_ = false; refreshSiren(); sink_(Config::Msg::kGasClear, true);
  }
}
```

Neden histerezis? Sensör değeri tam eşik etrafında (ör. 99-101) gürültüyle dalgalanırsa tek eşikli bir alarm saniyede birkaç kez açılıp kapanır, siren kesik kesik öter ve olay kaydı dolar. İki eşik (açma 100, kapama 80) arasında 20 birimlik bir "ölü bant" bırakarak durum kararlı tutulur.

Neden gaz alarmı durumdan bağımsız? Gaz/duman, kimse evde değilken de evdeyken de tehlikelidir. Hırsız alarmı ise yalnızca sistem kurulduğunda anlamlıdır. Bu yüzden gaz mantığı state makinesinin dışında tutulmuştur.

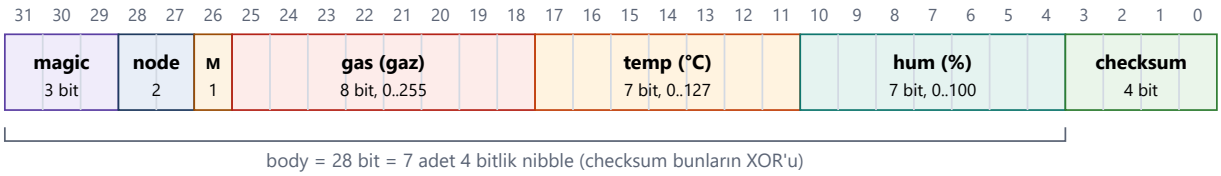
Neden Arming (çıkış süresi)? Sistemi kuran kişinin odadan çıkması için 10 sn verilir; bu sürede kendi hareketi alarmı tetiklemez.

> **Dikkat:** Kodda "giriş süresi" (entry delay) yok: Armed durumdayken hareket algılanır algılanmaz Triggered olur. Ayrıca kurma (`A`) kimlik doğrulama istemez, çözme ister. Bu, birçok ticari alarm panelinde de olan bilinçli bir tercih olabilir ama kodda açıklanmamıştır.

6 433 MHz RF protokolü (`RfPacket`)

`libraries/RfPacket/src/RfPacket.h` iki kartın da derlediği tek başlık dosyalı bir Arduino kütüphanesidir. Bir sensör okumasının tamamı **tek bir 32 bitlik rc-switch koduna** paketlenir.

```
libraries/RfPacket/src/RfPacket.h
// Layout (MSB -> LSB): magic(3) node(2) motion(1) gas(8) temp(7) hum(7) checksum(4)
struct SensorData { uint8_t nodeId; bool motion; uint8_t gasLevel; uint8_t temperature; uint8_t humidity;
};
constexpr unsigned int kBitLength = 32;
constexpr uint32_t kMagic = 0b101;
```



Örnek: düğüm 0, hareket var, gaz 45, 24 °C, %50 nem

101	00	1	00101101	0011000	0110010	1100
0b101	0	1	45	24	50	12

Gönderilen kod: **`0xA4B4C32C`** (ondalık 2763309868), 32 bit.

Checksum: body = 0xA4B4C32 → nibble'lar A,4,B,4,C,3,2; XOR = 0xC = 12.

Şekil 4. RfPacket bit yerleşimi (MSB solda) ve örnek kodlama.

6.1 Kodlama ve çözme

```
libraries/RfPacket/src/RfPacket.h
inline uint8_t checksum(uint32_t body) {
  uint8_t sum = 0;
  for (uint8_t i = 0; i < 7; ++i) sum ^= (body >> (i * 4)) & 0x0F;
  return sum;
}
inline bool decode(uint32_t raw, SensorData& out) {
  const uint32_t body = raw >> 4;
  if ((raw & 0x0F) != checksum(body) || (body >> 25) != kMagic) return false;
  ...
```

Alıcı tarafta (`NodeMonitor::poll`) bir paket şu dört kontrolden geçerse kabul edilir: (1) alınan bit uzunluğu 32, (2) checksum tutuyor, (3) üst 3 bit `101` , (4) düğüm kimliği < 4.

6.2 Tasarım kararları

Karar	Neden
433 MHz ISM bandı, ucuz OOK modüller	Lisans gerektirmez, modüller çok ucuzdur, duvar geçirgenliği 2,4 GHz'e göre iyidir; Nano'da Wi-Fi yoktur. Dezavantaj: tek yönlü, şifresiz, onay (ACK) yok.
rc-switch kütüphanesi	Hem AVR (Nano) hem ESP32'de çalışır, kesme tabanlı alım yapar, bir tamsayıyı (en çok 32 bit) doğrudan gönderir. Kullanımı tek satırdır: `rf.send(kod, 32)`. RadioHead (RH_ASK) daha esnek paketler sunar ama zamanlayıcı kullanımı ESP32/servo kütüphaneleriyle çakışabilir ve kurulumu daha ağırdır.
Tüm okuma tek 32 bitlik koda sığdırılır	rc-switch'in doğal birimi tek bir koddur; çok parçalı paketlerde parçalardan biri kaybolursa okuma bozulur. Alan genişlikleri gerçek aralıklara göre seçilmiştir (düğüm 0..3 → 2 bit, nem 0..100 → 7 bit).
Sihirli sayı (`101`)	Ortamdaki başka 433 MHz cihazların (kapı zili, uzaktan kumanda) kodlarını ayıklar.
4 bitlik XOR checksum	Gürültüden bozulmuş bitleri yakalar. 4 bit olduğu için rastgele bir kodun geçme olasılığı 1/16; sihirli sayı (1/8) ve bit uzunluğu kontrolüyle birlikte yanlış kabul olasılığı çok düşer. Güçlü değildir ama 32 bitlik bütçede makul bir uzlaşmadır.
Her kod 4 kez tekrarlanır (`kRepeatTransmit = 4`)	ACK olmadığı için güvenilirlik tekrarla sağlanır; en az birinin sağlam gelmesi yeterlidir.
Periyodik gönderim (2 sn) + 15 sn zaman aşımı	Alıcı, ardışık yaklaşık 7 paket kaçırılmadan düğümü çevrimdışı saymaz; tek tük kayıplar alarm üretmez.

> **Güvenlik sınırı:** Paketler şifresiz ve kimlik doğrulamasızdır; aynı formatta sahte paket üretmek ya da bandı karıştırmak (jamming) mümkündür. Bir öğrenci projesi için kabul edilebilir; gerçek bir üründe yuvarlanan kod (rolling code) veya şifreli bir radyo (ör. LoRa + AES) gerekir.

7 Oda düğümü (Arduino Nano)

`firmware/room_node/room_node.ino` yaklaşık 70 satırlık, sınıfsız bir sketch'tir. Görevi: sensörleri okumak, yerel LED/buzzer'ı sürmek ve okumayı RF ile göndermek.

Davranış	Ayrıntı (koddan)
Gönderim tetikleyicisi	Son gönderimden 2 sn (`kSendIntervalMs`) geçtiyse **veya** PIR yeni HIGH olduysa (yükselen kenar, `motionStarted`).
Gaz okuma	İlk 60 sn (`kGasWarmupMs`) 0 döner (MQ-2 ısıtıcısı ısınırken değer yüksek okunur). Sonra `analogRead(A0) >> 2` : 10 bitlik ADC (0..1023) 8 bite (0..255) indirilir.
DHT11	Sıcaklık 0..127, nem 0..100 aralığına sıkıştırılır. Okuma başarısızsa (NaN) son geçerli değer korunur.
Yerel buzzer	Gaz >= 100 (`Gas::kLocalAlarmLevel`) iken D5 HIGH. Merkezden bağımsız yerel uyarı.
LED'ler	Sarı: PIR anlık değeri (her döngüde). Kırmızı: gaz >= 100. Mavi: sıcaklık >= 30 °C (her gönderimde güncellenir).
Gönderim	Dahili LED gönderim süresince yanar; `rf.send(RfPacket::encode(reading), 32)` , 4 tekrar.

`firmware/room_node/room_node.ino`

```
const bool motion = digitalRead(Pin::kPir) == HIGH;
const bool motionStarted = motion && !lastMotion;
...
if (motionStarted || millis() - lastSend >= Timing::kSendIntervalMs) { ... transmit(); }
```

Neden hareket başlangıcında hemen gönderim? Periyodik 2 sn'lik gönderim tek başına kullanılsaydı, alarm tepkisi 2 sn'ye kadar gecikebilirdi. Yükselen kenarda anında gönderim, alarm gecikmesini radyo süresine indirir; periyodik gönderim de "hayatta" sinyali (heartbeat) işlevi görür.

Neden düğüm "aptal", merkez "akıllı"? Nano'nun belleği (2 KB RAM) ve iletişimi (tek yönlü) sınırlıdır. Karar mantığını merkezde tutmak, eşik ve kuralların tek yerde değiştirilmesini sağlar. Düğümde yalnızca yerel güvenlik için basit bir gaz buzzer'ı vardır.

Not: Düğümün yerel gaz buzzer'ında histerezis yoktur (tek eşik 100). `Timing::kLedBlinkMs` (30 ms) tanımlıdır ama kodda kullanılmamaktadır.

8 Yerel web paneli

`WebPanel`, ESP32'nin `WebServer` kütüphanesiyle port 80'de çalışır. Arayüz (`web_page.h`) koyu temalı tek bir HTML sayfasıdır; JavaScript 1,5 sn'de bir `/api/status` çağırır ve düğmeler `POST /api/*` gönderir.

8.1 Uç noktalar

Yöntem	Yol	İşlem	Yanıt
GET	`/`	Panel sayfası (PROGMEM)	HTML
GET	`/api/status`	İşlem yok	Durum JSON
POST	`/api/arm`	`alarm.arm("panel")`	Durum JSON
POST	`/api/disarm`	`alarm.disarm("panel")`	Durum JSON
POST	`/api/door`	`door.open()` + acil olay "Kapı panelden açıldı"	Durum JSON
POST	`/api/test`	`test.begin("panel")` (panel zaten kimlik doğruladığı için kart istenmez)	Durum JSON
POST	`/api/relay?on=1|0`	`relay.set(on)` + olay	Durum JSON

firmware/security_hub/WebPanel.cpp

```
// Every route is authenticated; API routes answer with the fresh status after acting.
void WebPanel::route(const char* path, HTTPMethod method, std::function<void()> action) {
  const bool isPage = strcmp(path, "/") == 0;
  server_.on(path, method, [this, action, isPage] {
    if (!authorized()) { server_.requestAuthentication(); return; }
    action();
    if (!isPage) sendStatus();
  });
}
```

> **Neden her komut sonrası durum dönülüyor?** Arayüz, düğmeye basıldıktan sonra ayrıca bir GET yapmadan yeni durumu hemen çizebilir. Tek bir `route()` yardımcı fonksiyonu kimlik doğrulamayı unutmayı imkânsız kılar.

8.2 Kimlik doğrulama: HTTP Basic Auth

Tarayıcı ilk istekte 401 + `WWW-Authenticate` alır ve kullanıcı adı/şifre sorar (`Secrets::kPanelUser`, `kPanelPassword`). Sonraki isteklerde `Authorization: Basic base64(kullanıcı:şifre)` başlığı gönderilir.

> **Sınır:** Bağlantı HTTP (TLS yok) olduğundan Basic Auth bilgisi yerel ağda düz metin sayılır (base64 şifreleme değildir). Panel yalnızca güvenilen yerel ağ için tasarlanmıştır; internete port yönlendirme yapılmamalıdır. Uzaktan erişim AWS üzerinden düşünülmüştür.

8.3 `/api/status` JSON alanları (`StatusReport.cpp`)

Alan	Tür	Anlamı
`state`	string	`disarmed` | `arming` | `armed` | `triggered`
`gasAlarm`	bool	Gaz alarmı etkin mi
`relay`	bool	Röle (fan) açık mı
`door`	bool	Kapı/vana açık mı (zamanlı açık veya `hold`)
`armLeft`	sayı (sn)	Kurulmaya kalan süre, yukarı yuvarlanmış
`armDelay`	sayı (sn)	10 (`kArmDelayMs / 1000`)
`gasLimit`	sayı	100 (`kGasAlarmLevel`)
`tempHigh`	sayı (°C)	30 (`kTempHighC`)
`uptime`	sayı (sn)	Açılıştan beri geçen süre
`nodes[]`	dizi	En az bir kez görülmüş düğümler: `id` , `online` , `temp` , `hum` , `gas` , `motion` , `age` (son paketten beri sn)
`log[]`	dizi	En yeni önce, en fazla 20 olay: `t` (saat), `m` (mesaj), `u` (acil mi)

```
{"state":"armed","gasAlarm":false,"relay":false,"door":false,"armLeft":0,"armDelay":10,
 "gasLimit":100,"tempHigh":30,"uptime":742,
 "nodes":[{"id":0,"online":true,"temp":24,"hum":50,"gas":45,"motion":false,"age":1}],
 "log":[{"t":"14:02:11","m":"Sistem kuruldu (tuş takımı)","u":true}]}
```

Örnek değerler açıklama amaçlıdır; alan adları ve türler koddan alınmıştır.

8.4 mDNS: `guvenlik.local`

`MDNS.begin("guvenlik")` ve `MDNS.addService("http", "tcp", 80)` ile cihaz yerel ağda `guvenlik.local` adıyla duyurulur. Kullanıcının IP adresini bilmesi gerekmez. mDNS bazı ağlarda veya eski Android sürümlerinde çalışmayabilir; bu yüzden IP adresi açılışta LCD'de gösterilir (README: açılmazsa `http://<IP>`).

> **Neden XSS koruması?** Sayfa olay mesajlarını `esc()` fonksiyonuyla HTML'den kaçırarak ekler. Tanımsız kart UID'si gibi dışarıdan gelen veriler olay kaydına girdiği için bu önemli bir alışkanlıktır.

9 PC kontrol merkezi

9.1 Merkez tarafı: `TelemetryClient`

firmware/security_hub/TelemetryClient.cpp

```
void TelemetryClient::update() {
  using namespace Config::Telemetry;
  const uint32_t now = millis();
  if (!kEnabled || WiFi.status() != WL_CONNECTED || static_cast<int32_t>(now - nextAttempt_) < 0) return;
  const bool ok = post(status_.toJson());
  nextAttempt_ = now + (ok ? kIntervalMs : kRetryMs);   // 5000 ms / 30000 ms
}
```

Ayar (`Config::Telemetry`)	Değer	Neden
`kServerUrl`	`http://192.168.1.91:8080/api/telemetry`	PC'nin yerel IP'si; ağ değişirse güncellenmeli.
`kIntervalMs`	5000	Grafik için yeterli çözünürlük, ağ/CPU yükü düşük.
`kRetryMs`	30000	**Geri çekilme:** PC kapalıyken her 5 sn'de bir 800 ms bloklamamak için.
`kTimeoutMs`	800	HTTP istemcisi bloklayıcıdır; kısa zaman aşımı ana döngüyü (kart, RF, siren) canlı tutar.
Başlık	`X-Api-Key: Secrets::kTelemetryKey`	Yerel ağdaki başka bir cihazın sahte veri yazmasını engelleyen paylaşılan anahtar.

9.2 PC tarafı: `pc-dashboard/server.py`

Yalnızca Python standart kütüphanesi: `http.server.ThreadingHTTPServer`, `sqlite3`, `json`, `threading`. Ayarlar `config.json`'dan okunur (şablon `config.example.json`: host `0.0.0.0`, port `8080`, veritabanı `telemetry.db`, `device_timeout_s` 20, `history_minutes` 60, `events_limit` 100).

Yöntem	Yol	Açıklama
POST	`/api/telemetry`	Anahtar yanlışsa 401; gövde 0 veya > 64 KB ise 400; geçersiz JSON 400; başarıda veriyi saklar, **204 No Content**.
GET	`/`	`index.html` paneli.
GET	`/api/latest`	Bellekteki son anlık görüntü, yaşı ve `online` (yaş < 20 sn).
GET	`/api/history?minutes=N`	Son N dakikanın örnekleri (en fazla 24 saat).
GET	`/api/events`	Son 100 olay, en yeni önce.

Panel sayfası `/api/latest`'i 2 sn, `/api/events`'i 5 sn, `/api/history?minutes=60`'ı 15 sn'de bir sorar.

SQLite şeması

pc-dashboard/server.py

```
CREATE TABLE IF NOT EXISTS samples (
    ts REAL NOT NULL, node INTEGER NOT NULL, temp INTEGER, hum INTEGER,
    gas INTEGER, motion INTEGER, state TEXT
);
CREATE INDEX IF NOT EXISTS idx_samples_ts ON samples (ts);
CREATE TABLE IF NOT EXISTS events (
    id INTEGER PRIMARY KEY AUTOINCREMENT, ts REAL NOT NULL, day TEXT NOT NULL,
    device_time TEXT NOT NULL, message TEXT NOT NULL, urgent INTEGER NOT NULL,
    UNIQUE (day, device_time, message)
);
```

Neden UNIQUE ile olay ayıklama? Merkez her POST'ta son 20 olayın tamamını yeniden gönderir (hangisinin daha önce iletildiğini takip etmez; bu merkezi basit ve durumsuz tutar). Sunucu `INSERT OR IGNORE` kullanır; `(day, device_time, message)` üçlüsü aynı olan kayıt ikinci kez eklenmez. `day`, PC'nin tarihidir; çünkü cihaz yalnızca saat (`SS:DD:ss`) gönderir ve ertesi gün aynı saatte aynı mesaj tekrar yazılabilmelidir.

Neden `samples` üzerinde `ts` indeksi? Geçmiş sorgusu `WHERE ts >= ?` ile zaman aralığı ister; 5 sn'de bir satır eklendiğinde günde yaklaşık 17 bin satır olur, indeks sorguyu hızlı tutar.

Neden `_db_lock`? `ThreadingHTTPServer` her isteği ayrı iş parçacığında işler; yazma işlemleri kilitle sıraya sokularak SQLite'ın "database is locked" hatası önlenir.

Sınır: Cihaz saati NTP almadıysa zaman damgası `+123s` biçimindedir; yeniden başlatmadan sonra aynı damga ve mesaj tekrar gelirse o olay yanlışlıkla "kopya" sayılıp atlanabilir. Aynı saniyede iki kez aynı mesaj da tek kayda düşer.

10 AWS IoT Core bulut katmanı

Bulut kısmı kuruludur ve tamamen kodla tanımlanmıştır: `cloud/template.yaml` (CloudFormation), `cloud/lambda/decide.py` (karar fonksiyonu), `cloud/deploy.ps1` / `destroy.ps1` (betikler) ve `cloud/config.json` (parametreler).

`cloud/config.json`	Değer
`region`	`eu-central-1` (Frankfurt)
`stack_name`	`iot-guvenlik`
`thing_name`	`hub01`
`topic_prefix`	`factory`
`thresholds`	`temp_on` 35, `temp_off` 32, `gas_on` 100, `gas_off` 80
`retention_days`	7

10.1 Kimlik ve bağlantı: Thing, X.509, MQTT/TLS

- **Thing** `hub01` : AWS IoT'de cihazın kaydı. Device Shadow bu ada bağlıdır.
- **X.509 cihaz sertifikası** ve özel anahtar: `deploy.ps1` `create-keys-and-certificate --set-as-active` ile üretir, Thing'e ve IoT politikasına bağlar. Cihaz bunlarla **karşılıklı TLS** (mTLS) yapar: sunucu cihazı sertifikasından tanır, şifre gerekmez.
- **Amazon Root CA 1**: cihaz, bağlandığı sunucunun gerçekten AWS olduğunu bu kök sertifikayla doğrular (`net_.setCACert`).
- **Port 8883** (`Config::Cloud::kPort`): MQTT over TLS standart portu. Uç nokta `iot:Data-ATS` tipindedir.
- **İstemci kimliği** = thing adı (`mqtt_.connect(AwsSecrets::kThingName)`); politika başka bir kimliğe izin vermez.
- **Saat şartı:** TLS sertifika geçerlilik tarihlerini kontrol eder; bu yüzden `time(nullptr) < kMinValidEpoch` (1700000000, Kasım 2023) iken bağlanma denenmez, NTP beklenir.
- Bağlantı kopukken 30 sn'de bir yeniden denenir (`kRetryMs`), keep-alive 60 sn, MQTT tampon 4096 bayt (olay kayıtlı durum JSON'u tek pakete sığsın diye).

`firmware/security_hub/CloudLink.cpp`

```
net_.setCACert(AwsSecrets::kRootCa);
net_.setCertificate(AwsSecrets::kDeviceCert);
net_.setPrivateKey(AwsSecrets::kPrivateKey);
mqtt_.setServer(AwsSecrets::kEndpoint, Config::Cloud::kPort);   // 8883
```

> **Neden sertifika, neden şifre değil?** Özel anahtar cihazdan hiç çıkmaz; sunucuya yalnızca imzayla sahip olduğu kanıtlanır. Sertifika tek tek iptal edilebilir (`update-certificate --new-status INACTIVE`), bir cihaz çalınsa diğerleri etkilenmez. AWS IoT Core cihaz kimliği için bu yöntemi esas alır.

10.2 Konu (topic) yapısı

Konu	Yön	İçerik	Sıklık
`factory/hub01/telemetry`	Cihaz → bulut	`StatusReport::toJson()` (yerel panelle aynı JSON)	5 sn
`factory/hub01/events`	Cihaz → bulut	Her olay ayrı mesaj: `{"t","m","u"}`, eskiden yeniye	Olay oluştukça
`$aws/things/hub01/shadow/update`	Cihaz → bulut	`{"state":{"reported":{"fan":"on|off","valve":"open|closed"}}}`	Bağlanınca ve her delta sonrası
`$aws/things/hub01/shadow/update/delta`	Bulut → cihaz	`{"state":{"fan":"on","valve":"open"},...}` (desired ≠ reported farkı)	Karar değişince

> **Neden `önek/thing/tür` hiyerarşisi?** IoT Rules `factory/+/telemetry` joker karakteriyle tüm cihazları tek kuralla dinler ve `topic(2)` ile cihaz adını konudan çıkarır. Politika da `factory/${thing}/*` ile her cihazı kendi alt ağacına hapseder. Yeni cihaz eklemek kural değiştirmeyi gerektirmez.

PubSubClient yalnızca QoS 0 ile yayın yapar; delta aboneliği QoS 1 ile istenir (`mqtt_.subscribe(shadow, 1)`).

10.3 IoT politikası: en az yetki

cloud/template.yaml (DevicePolicy, kısaltılmış)

```
iot:Connect   → client/${iot:Connection.Thing.ThingName}
iot:Publish   → topic/${TopicPrefix}/${iot:Connection.Thing.ThingName}/*
                topic/$aws/things/${iot:Connection.Thing.ThingName}/shadow/update
iot:Subscribe → topicfilter/$aws/things/${iot:Connection.Thing.ThingName}/shadow/update/delta
iot:Receive   → topic/$aws/things/${iot:Connection.Thing.ThingName}/shadow/update/delta
```

> **Neden politika değişkenleri?** `${iot:Connection.Thing.ThingName}`, bağlanan sertifikaya iliştirilmiş Thing'in adıyla çalışma anında değiştirilir. Böylece tek bir politika tüm cihazlara takılabilir ama her cihaz yalnızca kendi istemci kimliğini, konularını ve Shadow'unu kullanabilir. Ele geçirilmiş bir cihaz diğerlerinin konularına yazamaz veya başka bir cihazın Shadow komutlarını okuyamaz. Cihaz `shadow/get` veya başka bir konuya abone olamaz; yalnızca ihtiyaç duyduğu dört işlem izinlidir.

10.4 IoT Rules ve DynamoDB

cloud/template.yaml (TelemetryRule)

```
SELECT topic(2) AS device, timestamp() AS ts,
floor(timestamp() / 1000) + ${RetentionDays} * 86400 AS expires,
state, gasAlarm, relay, door, uptime, nodes
FROM '${TopicPrefix}/+/telemetry'
→ DynamoDBv2 PutItem (TelemetryTable)  +  Lambda (DecisionFunction)
```

cloud/template.yaml (EventsRule)

```
SELECT topic(2) AS device, timestamp() AS ts,
floor(timestamp() / 1000) + ${RetentionDays} * 86400 AS expires,
t AS deviceTime, m AS message, u AS urgent
FROM '${TopicPrefix}/+/events'
→ DynamoDBv2 PutItem (EventsTable)
```

Tablo	Anahtar	Faturalama	TTL
`TelemetryTable`	`device` (S, bölüm anahtarı) + `ts` (N, sıralama anahtarı, ms)	`PAY_PER_REQUEST`	`expires` özniteliği (epoch sn)
`EventsTable`	aynı	`PAY_PER_REQUEST`	`expires`

Tablo adları şablonda verilmediği için CloudFormation tarafından otomatik üretilir (yığın çıktıları `TelemetryTableName`, `EventsTableName`).

Neden kural motoru? Cihaz yalnızca MQTT'ye yayın yapar; verinin nereye yazılacağını bilmez. Saklama ve işleme sunucu tarafında SQL benzeri bir kuralla, sunucu kodu yazmadan tanımlanır. `DynamoDBv2` eylemi seçilen her alanı ayrı öznitelik olarak yazar (iç içe `nodes` dizisi liste olarak saklanır). Kuralın DynamoDB'ye yazabilmesi için yalnızca `dynamodb:PutItem` izni olan ayrı bir IAM rolü (`RuleRole`) vardır.

Neden `device + ts` anahtarı ve TTL? "Bir cihazın belirli zaman aralığındaki verileri" sorgusu tek bölümde sıralı okuma olur. `expires` = şimdi + 7 gün; DynamoDB süresi dolan kayıtları ücretsiz olarak kendiliğinden siler, tablo sınırsız büyümez.

10.5 Lambda kararı ve histerezis

`decide.py` (Python 3.12, zaman aşımı 10 sn) her telemetri mesajında TelemetryRule tarafından çağrılır. Eşikler ortam değişkenleri olarak (`TEMP_ON`, `TEMP_OFF`, `GAS_ON`, `GAS_OFF`) `config.json`'dan gelir.

cloud/lambda/decide.py

```
def hazard(temp, gas, active):
    if active:
        return temp > TEMP_OFF or gas > GAS_OFF      # 32 °C / 80
    return temp >= TEMP_ON or gas >= GAS_ON          # 35 °C / 100

def handler(event, context):
    thing = event["device"]
    nodes = [n for n in event.get("nodes", []) if n.get("online")]
    if not nodes: return {"skipped": "no online nodes"}
    temp = max(n["temp"] for n in nodes); gas = max(n["gas"] for n in nodes)
    desired = desired_state(thing)               # GetThingShadow
    active = desired.get("fan") == "on"
    danger = hazard(temp, gas, active)
    if danger == active and "fan" in desired:    # değişiklik yok: Shadow'a yazma
        return {"changed": False, ...}
    state = {"fan": "on" if danger else "off", "valve": "open" if danger else "closed"}
    iot.update_thing_shadow(thingName=thing, payload=json.dumps({"state": {"desired": state}}))
    print(json.dumps(decision))                  # CloudWatch Logs
```

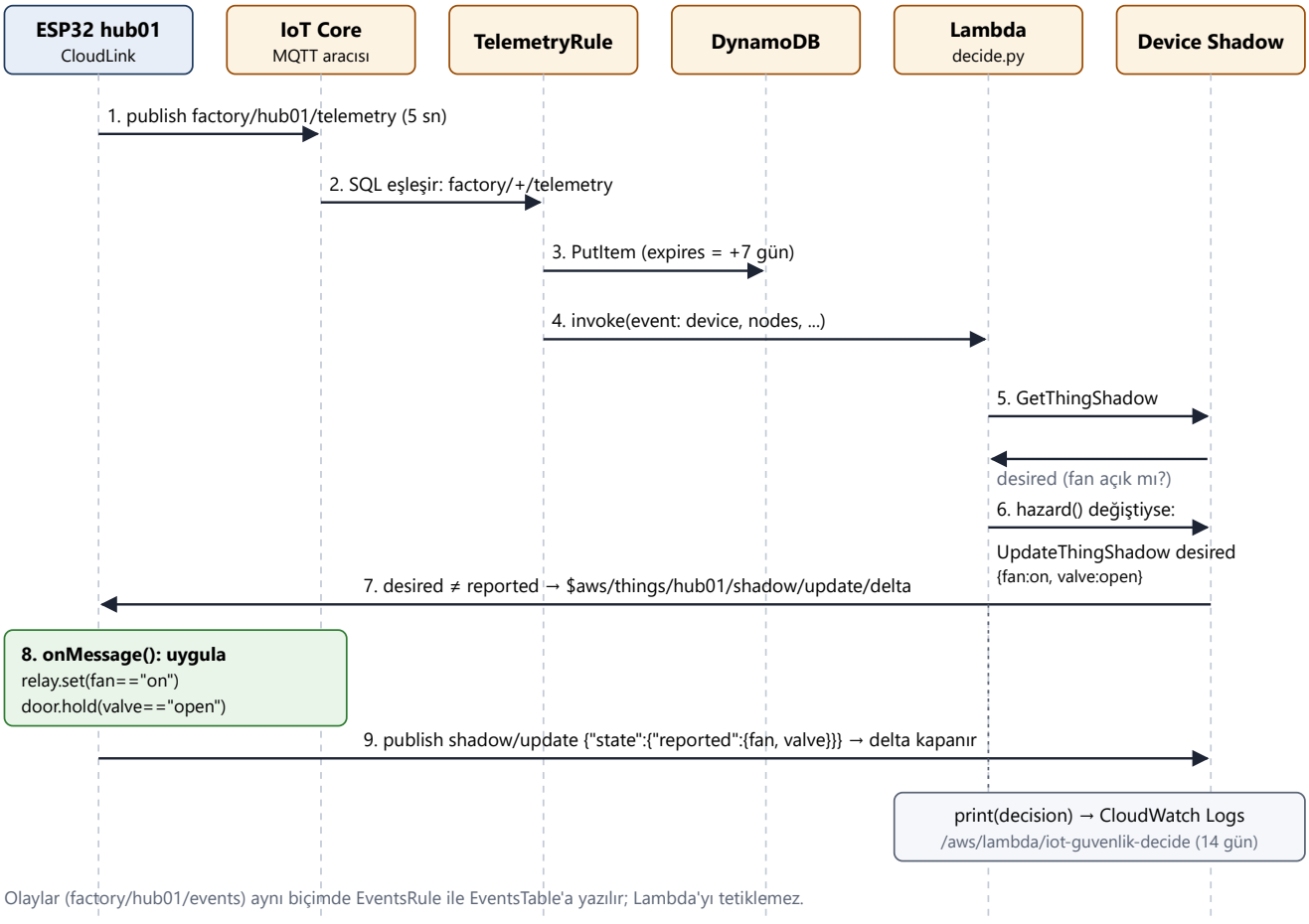
Mevcut durum	Fanı açma koşulu	Fanı kapatma koşulu
Fan kapalı	en yüksek sıcaklık >= 35 °C **veya** en yüksek gaz >= 100	-
Fan açık	-	sıcaklık <= 32 °C **ve** gaz <= 80

> **Neden durum Shadow'dan okunuyor?** Lambda durumsuzdur; her çağrı bağımsız bir konteynerde çalışabilir. "Fan şu an açık mı?" bilgisi (histerezis için gerekli) kalıcı bir yerde tutulmalıdır; Shadow'un `desired` bölümü bu hafızadır. Yalnızca karar değiştiğinde yazılması gereksiz Shadow güncellemelerini ve cihaz mesajlarını önler.

Lambda'nın yetkisi `DecisionRole` ile yalnızca bu Thing üzerinde `iot:GetThingShadow` ve `iot:UpdateThingShadow` ile sınırlıdır; loglar için `AWSLambdaBasicExecutionRole` eklenmiştir. IoT'nin fonksiyonu çağırabilmesi `DecisionInvokePermission` ile yalnızca bu kurala (`SourceArn`) ve bu hesaba verilmiştir.

10.6 Device Shadow ile komut akışı

Shadow, AWS'de saklanan JSON bir "dijital ikiz"dir: `desired` (olması istenen), `reported` (cihazın bildirdiği gerçek) ve ikisi farklıysa AWS'nin hesapladığı `delta` . Cihaz çevrimdışıyken verilen komut kaybolmaz; cihaz bağlanınca durumunu bildirir ve fark varsa delta alır.



Şekil 5. Telemetriden aktüatöre bulut karar döngüsü (sıralama diyagramı).

`firmware/security_hub/CloudLink.cpp`

```
// Shadow delta: {"state":{"fan":"on","valve":"open"},...}
void CloudLink::onMessage(char*, uint8_t* payload, unsigned int length) {
  JsonDocument doc;
  if (deserializeJson(doc, payload, length)) return;
  JsonObject state = doc["state"];
  if (state["fan"].is<const char*>())   actuators_.setFan(strcmp(state["fan"], "on") == 0);
  if (state["valve"].is<const char*>()) actuators_.setValve(strcmp(state["valve"], "open") == 0);
  reportState();
}
```

> **Neden doğrudan MQTT komutu değil de Shadow?** Doğrudan bir "fanı aç" mesajı cihaz o an çevrimdışıysa kaybolur. Shadow istenen durumu saklar; cihaz yeniden bağlandığında `connect()` içinde önce `reportState()` çağırır, fark varsa AWS delta gönderir. Ayrıca bulut tarafı cihazın gerçek durumunu (`reported`) her an okuyabilir. Komut "olay" değil "istenen durum" olarak modellenir; bu idempotenttir (aynı komutun iki kez gelmesi zarar vermez).

Yeniden bağlanmada olaylar: `connect()` , `publishedEvents_` sayacını tamponda hâlâ duran olayların başına çeker; böylece bağlantı kesikken oluşan (en fazla 20) olay bağlanınca eskiden yeniye gönderilir. Yayın başarısız olursa kalanlar sonraki turda denenir.

10.7 Dağıtım ve kaldırma betikleri

`deploy.ps1` (tekrar çalıştırılabilir)

1. `aws cloudformation deploy` : `template.yaml` , eşik ve isim parametreleri `config.json` 'dan, `CAPABILITY_IAM` .
2. Lambda kodu: `decide.py` zip'lenip `update-function-code` ile yüklenir (şablondaki kod yer tutucudur; tek doğruluk kaynağı `decide.py`).
3. Thing yoksa `create-thing` .
4. Sertifika yoksa `create-keys-and-certificate --set-as-active` ; dosyalar `cloud/.certs/` 'e; politika ve Thing'e bağlama.
5. `AmazonRootCA1.pem` indirilir (yoksa).
6. `describe-endpoint --endpoint-type iot:Data-ATS` ; uç nokta, thing adı, önek ve üç PEM `firmware/security_hub/aws_secrets.h` dosyasına yazılır ("do not commit").

`destroy.ps1`

1. Sertifikayı Thing'den ve politikadan ayırır.
2. Sertifikayı `INACTIVE` yapar ve siler.
3. Thing Shadow'unu siler.
4. Thing'i siler.
5. CloudFormation yığınını siler ve tamamlanmasını bekler (tablolar, Lambda, roller, kurallar, politika, log grubu).
6. Yerel `cloud/.certs/` ve `aws_secrets.h` dosyalarını siler.

> **Neden altyapı kod olarak (IaC)?** Konsolda elle tıklayarak kurulan bir sistem tekrarlanamaz ve unutulur. Şablon, tüm kaynakları ve izinleri gözden geçirilebilir tek bir dosyada toplar; `destroy.ps1` ile her şey tek komutla kaldırılır, unutulan kaynak maliyet üretmez. Sertifika ve Thing CloudFormation dışında, betikle yönetilir; çünkü özel anahtarın yerel dosyaya yazılması gerekir.

10.8 Yerel güvenlik yedeği ve maliyet kontrolü

İnternet yokken ne olur? `CloudLink::update()` Wi-Fi yoksa veya bağlantı kurulamazsa hemen döner ve 30 sn'de bir dener; TLS el sıkışması ana döngüyü ancak deneme anında kısa süre tutar. Kapı erişimi, hırsız alarmı, gaz alarmı, siren, LED'ler, LCD ve yerel web paneli tamamen ESP32 üzerinde çalıştığı için etkilenmez. Wi-Fi hiç yoksa bile (`kWifiFailed`) yerel işlevler devam eder; düğümün kendi gaz buzzer'ı da merkezden bağımsızdır.

> **Önemli ayrım:** Fan (röle) ve vana (servo) **otomatik** kontrolü yalnızca bulut Lambda'sındadır. İnternet kesikken yüksek gaz sireni çalar, ancak fan kendiliğinden açılmaz (panelden elle açılabilir). Merkezde gaz alarmında röleyi yerel olarak süren bir kod yoktur.

Maliyet önlemi	Nerede
DynamoDB `PAY_PER_REQUEST` (boşta ücret yok)	`template.yaml`
TTL ile 7 gün sonra otomatik silme (`retention_days`)	`template.yaml` , `config.json`
Lambda log grubu 14 gün saklama	`DecisionLogs`
Lambda yalnızca karar değişince Shadow'a yazar	`decide.py`
Telemetri 5 sn (dakikada 12 mesaj); olaylar yalnızca oluştukça	`config.h`
Tek komutla tüm kaynakların kaldırılması	`destroy.ps1`
Bütçe alarmı (AWS Budgets)	**Depoda tanımlı değil.** Konsolda elle kurulduysa koddan doğrulanamaz.

11 Teknoloji seçimleri tablosu

Teknoloji	Nerede	Neden seçildi	Alternatif ve karşılaştırma
ESP32-S3	Merkez	Dahili Wi-Fi, TLS için yeterli RAM ve hız, çok GPIO, Arduino desteği.	Raspberry Pi: Linux, daha güçlü ama açılışı yavaş, gerçek zamanlı değil, pahalı.
Arduino Nano	Düğüm	Ucuz, 5 V lojik (sensörlerle doğrudan uyumlu), basit.	İkinci bir ESP32 (ESP-NOW ile): daha yetenekli ama daha fazla güç ve maliyet.
rc-switch	RF	İki platformda da çalışır, 32 bitlik tek kod gönderimi basit.	RadioHead RH_ASK: paket, uzunluk, CRC sunar; daha ağır, zamanlayıcı çakışma riski.
HTTP POST	Merkez → PC	PC tarafında aracı (broker) kurmadan, stdlib ile alınabilir; hata ayıklaması kolay (curl, tarayıcı).	MQTT: yerelde Mosquitto aracısı gerekir; çift yönlü ve hafif ama ek kurulum.
MQTT (PubSubClient)	Merkez ↔ AWS	AWS IoT Core'un yerel protokolü; kalıcı bağlantı üzerinden çift yönlü (Shadow delta için şart), küçük başlık. PubSubClient küçük, Arduino'da yaygın, `WiFiClientSecure` ile TLS.	HTTPS ile AWS IoT'ye yalnızca yayın yapılabilir, komut almak için yoklama gerekir. Alternatif kütüphane: AWS IoT Device SDK, arduino-mqtt.
ArduinoJson	StatusReport, CloudLink	Belleği verimli, hem üretme hem ayrıştırma; elle dize birleştirmede kaçış hataları olmaz.	Elle `String` birleştirme: hataya açık. cJSON: C API, daha zahmetli.
Python stdlib (`http.server`)	PC sunucusu	Kurulum gerektirmez (`pip install` yok), tek dosya, öğretici.	FastAPI: otomatik doğrulama, OpenAPI dokümanı, async; ama bağımlılık kurulumu gerekir. Bu boyutta gereksiz.
SQLite	PC	Sunucusuz, tek dosya, SQL; yerel geçmiş için ideal.	DynamoDB: bulutta, ölçeklenir, yönetilen; ama internet ve hesap gerekir. Burada ikisi de var: yerel ve bulut kopya.
DynamoDB	AWS	Sunucusuz, isteğe göre ücret, TTL, IoT Rules ile doğrudan entegrasyon.	Timestream (zaman serisi, daha pahalı), RDS (sürekli çalışan sunucu), S3 + Athena (toplu analiz).
Lambda	AWS karar	Olay başına çalışır, sunucu yönetimi yok, IoT kuralı doğrudan tetikler.	Sürekli çalışan EC2: boşta da ücret. IoT Events: durum makinesi servisi, daha karmaşık.
Device Shadow	AWS komut	Çevrimdışı cihaza komut saklama, istenen/gerçek durum ayrımı.	Doğrudan komut konusu: basit ama kayıp riski. IoT Jobs: uzun süreli işler için.
CloudFormation	AWS kurulum	Tekrarlanabilir, gözden geçirilebilir, tek komutla silinebilir; ek araç gerektirmez (AWS CLI yeterli).	Elle konsol kurulumu: hızlı başlar ama belgesiz ve tekrarlanamaz. Terraform/CDK: güçlü ama ek araç.
mDNS	Yerel ağ	IP bilmeden `guvenlik.local` .	Yönlendiricide statik IP / DHCP rezervasyonu.

HTTP Basic Auth	Yerel panel	WebServer'da hazır, tarayıcı yerel destekler, oturum yönetimi gerekmez.	Oturum çerezi + giriş formu, ya da HTTPS + token: daha güvenli, ESP32'de daha zahmetli.

12 Donanım sorunları ve öğrenilen dersler

Aşağıdaki maddeler README "Güç" bölümü, breadboard rehberi notları ve kod yorumlarından derlenmiştir.

Belirti	Neden	Çözüm
LCD bozuk karakter yazıyor, RF alıcı ve servo çalışmıyor	Güç modülünün "5V OFF 3.3" jumper'ları 3,3 V konumunda (fabrikadan böyle gelebilir).	Her iki jumper'ı da 5V tarafına tak.
Servo dönmüyor, yalnızca tıkırdıyor; ESP32 yeniden başlıyor	Besleme yetersiz: 9 V pil veya 700 mA USB servo + röleyi taşıyamaz.	12 V, en az 2 A adaptör (bu kurulumda 3 A).
Breadboard'un bir yarısındaki modüllere güç gitmiyor	830 delikli kartlarda kenar güç hatları ortadan bölünmüş.	Dört hatta da 29-34 arası köprü kablosu.
Röle hiç kapanmıyor (sürekli çekili)	Aktif düşük 5 V modül, 3,3 V HIGH'ı "kapalı" saymıyor.	`OUTPUT_OPEN_DRAIN` : pin bırakılır, modülün pull-up'ı 5 V'a çeker (`Relay.h`).
ESP32'ye 5Vin'den güç verilemiyor	YD-ESP32-S3 klon kartında 5Vin pini, arkadaki IN-OUT pedi lehimlenmeden bağlı değil.	ESP32 USB'den beslenir; 3V3/5V pinleri güç modülüne bağlanmaz.
Tuşlar yanlış karakter veriyor	Tuş takımı şerit kablosu ters takılı.	Ölçülen haritaya göre `keypadRows` / `keypadCols` dizileri değiştirildi (`config.h`).
LCD boş kalıyor	I2C sırt kartının adresi farklı.	`Lcd::kAddress` 0x27 yerine 0x3F dene.
Açılıştan hemen sonra gaz alarmı	MQ-2 ısıtıcısı ısınana kadar yüksek okur.	Düğüm ilk 60 sn gaz değerini 0 gönderir (`kGasWarmupMs`); eşiği ısınma sonrası panel değerine göre ayarla.
PIR sürekli tetikli kalıyor	Modülün gecikme (süre) trimpotu uzun ayarlı.	İki turuncu trimpotu saat yönünün tersine sonuna kadar çevir (süre ~3 sn).
Servo dururken vızıldıyor	Sürekli PWM ile pozisyonu korumaya çalışıyor.	Hareketten 700 ms sonra `servo_.release()` (`kServoSettleMs`).
RF menzili kısa	Antensiz alıcı.	Alıcının ANT deliğine 17,3 cm (433 MHz'in çeyrek dalga boyu) düz tel.
ESP32 pinini 5 V sinyalden koruma	RF alıcı 5 V çıkış verir.	4,7 kΩ / 10 kΩ bölücü (≈3,4 V).
Nano yüklenmiyor	Klon kartta eski önyükleyici.	Arduino IDE: İşlemci "ATmega328P (Old Bootloader)".
AWS'ye bağlanamıyor (açılışta)	TLS saat ister; NTP gelmeden sertifika "geçerli değil" görünür.	Saat `kMinValidEpoch` 'u geçene kadar bağlantı denenmez.

> **Genel ders:** Gömülü sistem hatalarının büyük kısmı yazılımdan değil güçten (yetersiz akım, yanlış gerilim seviyesi, ortak toprak eksikliği) kaynaklanır. Belirti çoğu zaman yanıltıcıdır (LCD bozuk yazıyor, ama sorun güç jumper'ı). Önce besleme ölçülmelidir.

13 Sınav ve sunum soruları

1. Bu sistemde kenar (edge) bilişim nerede ve neden önemli?

ESP32 merkez, alarm ve erişim kararlarını buluta sormadan kendisi verir. İnternet kesilse veya gecikse de güvenlik işlevleri çalışır; bulut yalnızca gecikmeye dayanıklı fan/vana kararını verir.

2. Neden `delay()` kullanılmıyor?

`delay()` işlemciyi bloke eder; bu sürede kart, tuş, RF paketi ve web isteği kaçırılır. Bunun yerine her sınıf bir bitiş zamanı saklar ve `update()` içinde `millis()` ile kontrol eder.

3. Gaz alarmında histerezis nedir, değerleri nelerdir?

Açma ve kapama için iki ayrı eşik: merkezde 100'de açılır, 80'de kapanır. Lambda'da fan 35 °C/100 gazda açılır, 32 °C ve 80 gazın altına inince kapanır. Eşik çevresindeki gürültünün sürekli aç/kapa yaptırmasını önler.

4. RF paketinin yapısını anlatın.

32 bit: 3 bit sihirli sayı (101), 2 bit düğüm, 1 bit hareket, 8 bit gaz, 7 bit sıcaklık, 7 bit nem, 4 bit checksum (28 bitlik gövdenin 7 nibble'ının XOR'u).

5. 433 MHz bağlantının zayıf yönleri nelerdir?

Tek yönlü (ACK yok), şifresiz, kimlik doğrulamasız, karıştırmaya (jamming) ve tekrar saldırısına açık. Güvenilirlik 4 kez tekrar ve 15 sn zaman aşımıyla sağlanıyor.

6. Röle neden open-drain sürülüyor?

Aktif düşük 5 V röle modülü kapanmak için girişin 5 V'a yakın olmasını ister; ESP32'nin 3,3 V'u yetmez. Open-drain'de pin bırakılır ve modülün pull-up'ı girişi 5 V'a çeker.

7. RF alıcı neden gerilim bölücüyle bağlanıyor?

Alıcı 5 V sinyal verir, ESP32 pinleri 3,3 V'tur. 4,7k/10k bölücü 5 V'u yaklaşık 3,4 V'a indirir.

8. Cihaz AWS'ye nasıl kimlik kanıtlar?

X.509 cihaz sertifikası ve özel anahtarla karşılıklı TLS (port 8883). Sunucu da Amazon Root CA 1 ile doğrulanır. İstemci kimliği thing adı olmalıdır.

9. IoT politikasında "en az yetki" nasıl sağlanıyor?

`${iot:Connection.Thing.ThingName}` değişkeniyle her cihaz yalnızca kendi istemci kimliğiyle bağlanabilir, yalnızca `factory/<kendi adı>/*` ve kendi Shadow update konusuna yayın yapabilir, yalnızca kendi delta konusuna abone olabilir.

10. Device Shadow'da desired, reported ve delta nedir?

desired: bulutun istediği durum (Lambda yazar). reported: cihazın bildirdiği gerçek durum. delta: ikisi farklıysa AWS'nin cihaza gönderdiği fark. Cihaz çevrimdışıyken de komut saklanır.

11. Lambda neden Shadow'u okuyup ancak değişiklikte yazıyor?

Lambda durumsuzdur; fanın şu anki durumunu (histerezis için) Shadow'dan öğrenir. Değişmeyen kararı yazmamak gereksiz mesaj ve maliyeti önler.

12. DynamoDB'de TTL ne işe yarar?

Her kayda `expires` = şimdi + 7 gün yazılır; süre dolunca DynamoDB kaydı ücretsiz siler. Depolama ve maliyet sınırsız büyümez.

13. PC sunucusu tekrar gelen olayları nasıl ayıklıyor?

Merkez her POST'ta son 20 olayı gönderir; SQLite'ta `UNIQUE(day, device_time, message)` kısıtı ve `INSERT OR IGNORE` ile aynı olay ikinci kez eklenmez.

14. PC kapalıyken merkez neden yavaşlamıyor?

HTTP isteği 800 ms'de zaman aşımına uğrar ve başarısız denemeden sonra 30 sn beklenir (geri çekilme). Böylece döngü en fazla 30 sn'de bir 0,8 sn bekler.

15. EventSink kullanmanın faydası nedir?

Olay üreten sınıflar (AlarmSystem, NodeMonitor, ...) olayların log'a mı Telegram'a mı gideceğini bilmez; tek bir fonksiyon (`report()`) yönlendirir. Bu gevşek bağlılık, yeni bir çıkış eklemeyi tek satıra indirir.

16. Neden CloudFormation?

Tüm bulut kaynakları ve izinler tek dosyada, tekrarlanabilir ve gözden geçirilebilir; `destroy.ps1` ile tamamen kaldırılabilir, unutulan kaynak kalmaz.

14 Sözlük

ACK (onay)
Alıcının mesajı aldığını gönderene bildirmesi. 433 MHz bağlantıda yoktur.

Aktif düşük (active-low)
Girişin LOW olduğunda "açık" anlamına geldiği sinyal mantığı. Röle modülü böyledir.

Captive portal
Bir Wi-Fi ağına bağlanınca tüm adreslerin bir kurulum/giriş sayfasına yönlendirilmesi.

Checksum (sağlama toplamı)
Verinin bozulup bozulmadığını anlamak için verilerden hesaplanan küçük değer. Burada 4 bitlik XOR.

CloudFormation
AWS kaynaklarını YAML/JSON şablonla tanımlayıp kuran altyapı-kod (IaC) servisi.

CloudWatch Logs
AWS'nin log toplama servisi; Lambda'nın `print` çıktıları buraya düşer.

Device Shadow
AWS IoT'de bir cihazın istenen (desired) ve bildirilen (reported) durumunu saklayan JSON belge; fark (delta) cihaza gönderilir.

DynamoDB
AWS'nin sunucusuz anahtar-değer / belge veritabanı.

Gerilim bölücü
Seri iki dirençle bir gerilimi orantılı olarak düşüren devre: $V_{out} = V_{in} \times R2 / (R1 + R2)$.

Histerezis
Açma ve kapama için farklı eşik kullanarak eşik çevresindeki titreşimi önleme.

I2C
İki telli (SDA veri, SCL saat) adreslemeli seri veri yolu. LCD bu yolla bağlıdır (adres 0x27).

IaC (Infrastructure as Code)
Altyapının elle değil, sürüm kontrolündeki kodla tanımlanması.

IoT Policy
Bir sertifikanın AWS IoT'de hangi işlemleri (Connect, Publish, Subscribe, Receive) hangi kaynaklarda yapabileceğini belirleyen JSON izin belgesi.

IoT Rule
MQTT mesajlarını SQL benzeri bir sorguyla süzüp DynamoDB, Lambda gibi hedeflere ileten AWS IoT kuralı.

ISM bandı
Lisanssız kullanılabilen endüstriyel/bilimsel/tıbbi radyo frekansları (433 MHz, 2,4 GHz).

JSON
Anahtar-değer tabanlı, insan tarafından okunabilir veri biçimi.

Lambda
AWS'nin olayla tetiklenen sunucusuz fonksiyon servisi.

mDNS
Yerel ağda DNS sunucusu olmadan `ad.local` biçiminde isim çözme (multicast DNS).

millis()
Arduino'da açılıştan beri geçen milisaniye; yaklaşık 49,7 günde taşar.

MQTT
: Yayınla/abone ol (publish/subscribe) modelli, hafif mesajlaşma protokolü. Mesajlar bir aracı (broker) üzerinden konulara (topic) gönderilir.

mTLS
: İki tarafın da sertifikayla kimlik kanıtladığı TLS.

NTP
: Ağ üzerinden saat eşitleme protokolü.

NVS
: ESP32'nin flash belleğinde anahtar-değer saklayan kalıcı depolama alanı.

OOK
: On-Off Keying: taşıyıcının varlığı/yokluğuyla bit gönderen basit modülasyon (ucuz 433 MHz modüller).

Open-drain
: Pinin yalnızca LOW'a çekebildiği, "HIGH" durumunda serbest (yüksek empedans) kaldığı çıkış modu; HIGH seviyesini harici pull-up belirler.

PEM
: Sertifika ve anahtarların `-----BEGIN ..------` satırlarıyla çevrili base64 metin biçimi.

Pull-up direnci
: Bir hattı boştayken besleme gerilimine çeken direnç.

PWM
: Darbe genişlik modülasyonu. Servo 50 Hz PWM'deki darbe genişliğine göre açı alır.

QoS
: MQTT teslim garantisi seviyesi: 0 (en fazla bir kez), 1 (en az bir kez), 2 (tam bir kez).

REST
: Kaynakların URL ile, işlemlerin HTTP yöntemleriyle (GET, POST) ifade edildiği API tarzı.

SPI
: Saat (SCK), MOSI, MISO ve seçme (SS) hatlı hızlı seri veri yolu. RC522 bu yolla bağlıdır.

SQLite
: Tek dosyada çalışan, sunucusuz ilişkisel veritabanı.

TLS
: Taşıma katmanı güvenliği: şifreleme, bütünlük ve sertifikayla kimlik doğrulama sağlar.

TTL (Time To Live)
: Bir kaydın yaşam süresi; DynamoDB'de süresi dolan kayıtlar otomatik silinir.

X.509
: Açık anahtar sertifikalarının standart biçimi; kimlik, açık anahtar ve imzalayan otoriteyi içerir.

15 Ek: Kodda gözlenen tutarsızlıklar

Bu belge hazırlanırken fark edilen ve sunumda sorulabilecek noktalar. Kod değiştirilmemiştir.

1. **Kurulum portalı bağlanmamış:** `DeviceSettings`, `SetupPortal`, `setup_page.h` ve `config.h` içindeki `Setup::`, `kSetupWindowMs`, `Keypad4x4::kSetup`, `kSetup*` mesajları mevcut ama `security_hub.ino` bunları kullanmıyor. `SetupPortal.cpp` yorumunda belirtilen `/panel` yolu `WebPanel`'de yok (panel `/`'de). `Cloud::kEnabled` yorumu "setup portal or aws_secrets.h" diyor; `CloudLink` yalnızca `aws_secrets.h` kullanıyor.
2. **Olası derleme sorunu:** `SetupPortal.cpp` `Secrets::kSetupApPassword` kullanıyor; bu sabit `secrets.example.h` şablonunda yok. Arduino IDE sketch klasöründeki tüm .cpp dosyalarını derlediği için, `secrets.h` bu sabiti içermiyorsa proje derlenmez (secrets.h bu çalışmada okunmadı, doğrulanamadı).
3. **Adaptör akımı:** README ve breadboard rehberi "12 V, en az 2 A" diyor; bu kurulumda 3 A kullanıldığı belirtiliyor. Çelişki değil ama belgeler güncellenebilir.
4. **README kütüphane listesi eksik:** AWS için gereken `PubSubClient` README kurulum adımında yok. README mimari şeması ve metni AWS katmanından hiç bahsetmiyor.
5. **Bulutta olay kopyaları:** `CloudLink::connect()` her yeniden bağlanmada tamponda duran olayları yeniden yayınlar. `EventsTable` anahtarı sunucu zaman damgası (`ts`) olduğu için daha önce iletilmiş olaylar DynamoDB'de yeni kayıt olarak çoğalabilir (PC tarafındaki gibi bir UNIQUE ayıklaması yok).
6. **Yerel otomatik fan yok:** Gaz alarmında fan/vana yalnızca Lambda kararıyla açılıyor; internet yoksa otomatik havalandırma devreye girmiyor (siren yine çalışıyor). Akıllı fabrika yorumunda bu bir eksik sayılabilir.
7. **Panel ve bulut aynı röleyi sürüyor:** Yerel paneldeki röle düğmesi ve Lambda'nın `fan` kararı aynı çıkışı kontrol ediyor; aralarında öncelik kuralı yok. Elle değişiklik Shadow `reported`'a yalnızca bir sonraki delta veya yeniden bağlanmada yansır.
8. **Bloklama istisnaları:** Bloklamayan döngü ilkesine üç istisna var: açılıştaki 15 sn'lik Wi-Fi bekleme döngüsü, `TelegramNotifier` (yorumuna göre mesaj başına ~1 sn, zaman aşımı 5 sn; varsayılan kapalı) ve MQTT/TLS bağlantı denemesi (süresi kodda sınırlanmamış).
9. **Sıcaklık eşikleri farklı:** Mavi LED 30 °C'de (`kTempHighC`), bulut fanı 35 °C'de açılıyor. Amaçları farklı (gösterge ve eylem) ama sunumda karıştırılmamalı.
10. **Kullanılmayan sabit:** `room_node/config.h` içindeki `Timing::kLedBlinkMs` kodda kullanılmıyor. Düğümün yerel gaz buzzer'ında histerezis yok.
11. **Giriş gecikmesi yok, kurma doğrulamasız:** Armed iken hareket anında alarm verir (yetkili kişinin şifre girmesi için süre yok); `A` tuşu kimlik doğrulamadan kurar.
12. **Telegram TLS doğrulaması kapalı:** `client.setInsecure()` ("demo project"), ortadaki adam saldırısına açık; özellik varsayılan kapalı.